

Boost.Spatial: A Generic N-Dimensional Sparse Spatial Hash Grid

High-Performance Neighbor Search with Zero-Overhead Abstractions

Alex Towell

PhD Student, Computer Science
Southern Illinois University Edwardsville
`lex@metafunctor.com`

November 11, 2025

Abstract

Spatial indexing is fundamental to many computational domains including game development, molecular dynamics, particle simulations, and computational geometry. Traditional approaches such as dense grids, octrees, and R-trees present trade-offs between memory efficiency, update performance, and query speed. We present `boost_sparse_spatial_hash`, a header-only C++20 library that provides a generic N-dimensional sparse spatial hash grid optimized for dynamic scenes with frequent updates. The library features three topology modes (bounded, toroidal, and infinite), Morton encoding for spatial locality, incremental updates that achieve 40x speedup over full rebuilds, and a custom small vector implementation that eliminates heap allocation overhead for typical cell sizes. Performance evaluations demonstrate throughput of 25M entities/second for grid construction, 400K queries/second for medium-radius queries, and 2.4M pairs/second for collision detection, with 5-40% improvement from small vector optimization and 23% from micro-optimizations. The design leverages C++20 concepts, compile-time polymorphism, and position accessor traits to achieve zero-overhead abstractions while maintaining type safety and extensibility. The library has been rigorously tested with 54 test cases covering 326 assertions and is ready for Boost submission.

1 Introduction

1.1 Motivation

Spatial indexing structures enable efficient proximity queries by organizing entities according to their spatial relationships. These queries are ubiquitous in computational applications: game engines perform collision detection among thousands of objects per frame, molecular dynamics simulations compute pairwise forces between millions of particles, crowd simulation systems coordinate agent behaviors based on local neighborhoods, and geographic information systems answer spatial proximity queries over large datasets.

The naive approach of testing all entity pairs scales as $O(n^2)$, becoming prohibitively expensive for even moderate entity counts. A simulation with 10,000 entities would require 50 million pairwise comparisons per frame. At 60 frames per second, this translates to 3 billion comparisons per second, far exceeding practical computational budgets.

Spatial indexing structures reduce this complexity by partitioning space and limiting queries to relevant regions. A well-designed spatial index transforms proximity queries from $O(n)$ to $O(k)$

where k represents the number of entities in the query region, typically orders of magnitude smaller than n .

1.2 Existing Approaches and Limitations

Several spatial indexing structures exist, each with distinct characteristics:

Dense uniform grids partition space into a regular lattice of cells and store entity references in each cell. Insertions and queries operate in $O(1)$ time, making them attractive for performance-critical applications. However, memory consumption scales with the total number of cells rather than occupied cells. For sparse worlds such as a $10,000^3$ simulation with 10 million particles, a dense grid would allocate approximately 6 terabytes of memory to store empty cells, a 60,000x overhead compared to actual data.

Hierarchical trees such as quadtrees, octrees, k-d trees, and R-trees organize space hierarchically, adapting resolution to data density. These structures achieve good memory efficiency and logarithmic query complexity. However, they require rebalancing or reconstruction when entities move, making them poorly suited for highly dynamic scenes. Tree traversal also introduces pointer indirection that can harm cache performance.

Hash-based spatial structures combine the constant-time operations of hash tables with spatial partitioning. Early work by Teschner et al. [1] demonstrated optimized spatial hashing for collision detection, achieving significant performance improvements over tree-based approaches. However, most implementations lack genericity, supporting only specific dimensions or entity types.

1.3 Our Approach

We present a sparse spatial hash grid that addresses these limitations through:

- **Sparse storage:** Only occupied cells consume memory, achieving 60,000x reduction for sparse worlds
- **Incremental updates:** Entities that remain in their cells require no hash table operations, achieving 40x speedup when 99% of entities are stationary
- **Generic programming:** Template parameters and concepts enable support for arbitrary dimensions, coordinate types, and entity types without runtime overhead
- **Topology modes:** First-class support for bounded, toroidal, and infinite topologies accommodates diverse application domains
- **Morton encoding:** Z-order curves provide spatial locality in hash table storage, improving cache utilization

The library targets Boost-quality standards with comprehensive testing, header-only distribution, and zero external dependencies beyond the C++20 standard library.

1.4 Contributions

This work makes the following contributions:

1. A generic N-dimensional sparse spatial hash implementation using C++20 concepts and compile-time polymorphism

2. An incremental update algorithm that tracks entity cell occupancy and updates only changed entities
3. Three topology modes (bounded, toroidal, infinite) with consistent APIs and performance characteristics
4. A custom small vector implementation that stores typical cell sizes (≤ 16 entities) inline, achieving 5-40% performance improvement by eliminating heap allocations
5. Targeted micro-optimizations including precomputed division reciprocals, manual loop unrolling, and optimized Morton encoding achieving 23% overall speedup
6. Comprehensive validation through 54 test cases covering 326 assertions, including correctness, exception safety, and limit testing
7. Performance evaluation demonstrating throughput of 25M entities/second build, 400K queries/second, 2.4M pairs/second collision detection

2 Design and Abstractions

2.1 Core Concept: Spatial Hashing

Spatial hashing partitions continuous space into a uniform grid of cells and maps each cell to a hash table bucket. Given an entity at position $\mathbf{p} = (p_0, p_1, \dots, p_{n-1})$ in an n -dimensional space, we compute its cell coordinates as:

$$c_i = \left\lfloor \frac{p_i + w_i/2}{s_i} \right\rfloor \quad (1)$$

where w_i is the world size in dimension i , s_i is the cell size in dimension i , and c_i is the integer cell coordinate. The offset by $w_i/2$ centers the grid at the origin.

The cell coordinate tuple $(c_0, c_1, \dots, c_{n-1})$ is hashed to produce a hash table key. All entities within the same cell map to the same bucket, enabling constant-time insertion and cell-based queries.

For a radius query centered at position $\mathbf{q}$ with radius r , we compute the cell radius in each dimension:

$$r_i = \left\lceil \frac{r}{s_i} \right\rceil \quad (2)$$

and iterate over all cells within the bounding box $[c_i - r_i, c_i + r_i]$ in each dimension, collecting entities from occupied cells.

2.2 Morton Encoding for Spatial Locality

Standard hash functions such as polynomial rolling hashes distribute keys uniformly across the hash table address space, destroying spatial locality. Nearby cells in space map to distant buckets in memory, resulting in poor cache utilization during neighborhood queries.

Morton encoding, also known as Z-order curves, preserves spatial locality by interleaving the bits of cell coordinates. For a 3D cell at coordinates (x, y, z) , the Morton code is constructed by interleaving bits:

$$\text{morton}(x, y, z) = \dots z_2 y_2 x_2 z_1 y_1 x_1 z_0 y_0 x_0 \quad (3)$$

where x_i, y_i, z_i represent the i -th bit of each coordinate. This construction ensures that cells close in Euclidean space have numerically similar hash keys, improving spatial coherence in hash table storage.

Our implementation provides optimized Morton encoding for 2D and 3D using bit spreading with precomputed masks, achieving 5-8% speedup over generic loop-based implementations:

```

1 inline uint64_t morton_encode_3d(int32_t x, int32_t y, int32_t z) {
2     auto spread = [](uint32_t n) -> uint64_t {
3         uint64_t x = n & 0xffffffff;
4         x = (x | x << 32) & 0x1f00000000ffff;
5         x = (x | x << 16) & 0x1f0000ff0000ff;
6         x = (x | x << 8) & 0x100f00f00f00f0f;
7         x = (x | x << 4) & 0x10c30c30c30c30c3;
8         x = (x | x << 2) & 0x1249249249249249;
9         return x;
10    };
11    return spread(x) | (spread(y) << 1) | (spread(z) << 2);
12 }

```

Listing 1: Optimized 3D Morton encoding using bit spreading

For dimensions greater than 3, the implementation falls back to a generic bit-interleaving loop that maintains correctness while sacrificing some performance.

2.3 Topology Modes

Real-world applications require different boundary behaviors. We provide three topology modes with consistent interfaces:

Bounded topology clamps entity coordinates to grid boundaries, appropriate for simulations with physical walls or fixed spatial extents. Cell coordinates are clamped to the range $[0, r_i - 1]$ where r_i is the grid resolution in dimension i .

Toroidal topology wraps coordinates at boundaries using modular arithmetic, creating periodic boundary conditions. This topology is essential for simulations that approximate infinite systems using finite computational domains, common in molecular dynamics and cosmological simulations. Cell coordinates are computed as:

$$c'_i = ((c_i \bmod r_i) + r_i) \bmod r_i \quad (4)$$

The double modulo operation handles negative coordinates correctly.

Infinite topology imposes no boundary constraints, allowing the grid to grow dynamically. This mode suits applications with unbounded spatial domains or unknown extents, such as exploratory simulations or procedurally generated worlds.

Topology selection occurs at grid construction and incurs no runtime overhead due to compile-time optimization through switch statements that compilers resolve to direct jumps.

2.4 Generic Programming with C++20 Concepts

The library design prioritizes generic programming to support diverse entity types, dimensions, and numeric precision without sacrificing performance or type safety.

2.4.1 Position Accessor Trait

Rather than constraining entity types to specific interfaces, we use a customization point through the `position_accessor` trait:

```
1 template<typename T, std::size_t Dims>
2 struct position_accessor {
3     static auto get(const T& obj, std::size_t dim);
4 };
5
6 // User specialization for custom types
7 template<>
8 struct position_accessor<Particle, 3> {
9     static float get(const Particle& p, std::size_t dim) {
10         switch(dim) {
11             case 0: return p.x;
12             case 1: return p.y;
13             case 2: return p.z;
14         }
15     }
16 };
```

Listing 2: Position accessor customization point

This design provides several advantages. Users maintain complete control over their entity types without inheritance or interface conformance. The trait enables support for third-party types that cannot be modified. Compilers can inline the accessor, eliminating abstraction overhead.

2.4.2 Concepts for Type Safety

We employ C++20 concepts to enforce compile-time constraints on template parameters:

```
1 template<typename T>
2 concept coordinate_type = std::is_arithmetic_v<T> &&
3                           std::totally_ordered<T>;
4
5 template<typename T, std::size_t Dims>
6 concept has_position = requires(const T& obj, std::size_t dim) {
7     { position_accessor<T, Dims>::get(obj, dim) }
8     -> coordinate_type;
9 };
```

Listing 3: Core concepts for type constraints

These concepts provide clear error messages when type requirements are violated and enable the compiler to perform better optimizations by understanding type constraints.

2.4.3 Compile-Time Dimension Optimization

Template parameters for dimension count enable compile-time specialization:

```
1 if constexpr (Dims == 2) {
2     // Optimized 2D path with manual unrolling
3     float x = position_accessor<EntityT, Dims>::get(entity, 0);
4     float y = position_accessor<EntityT, Dims>::get(entity, 1);
5     cell[0] = static_cast<int>(std::floor(x * inv_cell_size[0]));
6 }
```

```

6     cell[1] = static_cast<int>(std::floor(y * inv_cell_size[1]));
7 } else if constexpr (Dims == 3) {
8     // Optimized 3D path
9     // ...
10 } else {
11     // Generic N-dimensional fallback
12     for (std::size_t d = 0; d < Dims; ++d) {
13         // ...
14     }
15 }

```

Listing 4: Dimension-specific optimization example

The `if constexpr` construct eliminates unused branches at compile time, generating specialized code for common cases without runtime overhead.

2.5 API Design

The public interface balances ease of use with performance:

```

1 // Configuration
2 grid_config<3> cfg{
3     .cell_size = {10.0f, 10.0f, 10.0f},
4     .world_size = {1000.0f, 1000.0f, 1000.0f},
5     .topology_type = topology::toroidal
6 };
7
8 // Construction
9 sparse_spatial_hash<Particle, 3> grid(cfg);
10
11 // Full rebuild (initial construction)
12 grid.rebuild(particles);
13
14 // Incremental update (subsequent frames)
15 grid.update(particles);
16
17 // Radius query (returns entity indices)
18 auto neighbors = grid.query_radius(50.0f, x, y, z);
19
20 // Process all pairs within radius
21 grid.for_each_pair(particles, radius,
22     [](std::size_t i, std::size_t j) {
23         // Handle collision/interaction
24     });
25
26 // Statistics
27 auto stats = grid.stats();
28 std::cout << "Occupancy: " << stats.occupancy_ratio << "\n";

```

Listing 5: Core API example

The API supports C++20 ranges, enabling direct use with views and projections:

```

1 // Filter and build from subset
2 auto moving = particles
3     | std::views::filter([](auto& p) { return p.velocity > threshold; });

```

```
4 grid.rebuild(moving);
```

Listing 6: Range support example

2.6 Incremental Update Strategy

The incremental update algorithm exploits temporal coherence in dynamic scenes. In typical simulations, entities move small distances between frames, and the majority remain within their current cells.

The grid maintains a vector `entity_cells_` that maps each entity index to its current cell coordinates. During an update operation:

Algorithm 1 Incremental Update Algorithm

```

1: for each entity  $i$  in entities do
2:    $c_{\text{new}} \leftarrow$  compute cell coordinate for entity  $i$ 
3:    $c_{\text{old}} \leftarrow \text{entity\_cells\_}[i]$ 
4:   if  $c_{\text{new}} \neq c_{\text{old}}$  then
5:     remove  $i$  from hash bucket  $c_{\text{old}}$ 
6:     insert  $i$  into hash bucket  $c_{\text{new}}$ 
7:      $\text{entity\_cells\_}[i] \leftarrow c_{\text{new}}$ 
8:   end if
9: end for
```

For a scenario where 99% of entities remain in their cells (typical of small time steps), the update performs work proportional to $0.01n$ rather than n , achieving a 100x theoretical speedup. In practice, hash table operations and memory access patterns reduce this to approximately 40x observed speedup.

The removal operation uses swap-and-pop rather than `std::remove`, providing $O(1)$ removal instead of $O(m)$ where m is the bucket size:

```

1 for (std::size_t i = 0; i < bucket.size(); ++i) {
2   if (bucket[i] == entity_idx) {
3     if (i != bucket.size() - 1) {
4       bucket[i] = bucket.back(); // Swap
5     }
6     bucket.pop_back(); // Pop
7     break;
8   }
9 }
```

Listing 7: Swap-and-pop removal optimization

This optimization contributes 3-5% improvement on update operations.

3 Implementation Highlights

3.1 Hash Table Design

The implementation uses `std::unordered_map` as the underlying hash table, mapping cell coordinates to entity index lists. This design choice balances several factors:

- Standard library implementation provides reasonable performance across platforms
- Automatic memory management simplifies implementation
- Future work can substitute alternative hash tables (e.g., `boost::unordered_flat_map`) without API changes

Cell buckets employ a custom `small_vector<T, N>` implementation that stores $\leq N$ elements inline on the stack, falling back to heap allocation for larger buckets. With a default capacity of 16 elements, 80-90% of cells avoid heap allocation entirely. This optimization provides 5-40% performance improvement (40% on rebuild, 5-11% on updates and queries) while adding only $8 \times N$ bytes per cell (~ 256 bytes with $N=16$). The capacity is tunable via a template parameter, allowing users to optimize for their specific workload characteristics.

3.2 Performance Optimizations

We implemented five key optimizations that collectively achieve 23% geometric mean performance improvement:

3.2.1 Precomputed Division Reciprocals

Floating-point division exhibits 3-4x higher latency than multiplication on modern processors (10-15 cycles vs 3-5 cycles). Cell coordinate computation requires dividing by cell size for every entity:

$$c_i = \left\lfloor \frac{p_i + w_i/2}{s_i} \right\rfloor \quad (5)$$

We precompute reciprocals s_i^{-1} at grid construction and replace division with multiplication:

$$c_i = \lfloor (p_i + w_i/2) \cdot s_i^{-1} \rfloor \quad (6)$$

This optimization provides 8-12% improvement on build operations and 20-30% on queries, with negligible memory cost (12 bytes for 3D).

3.2.2 Manual Loop Unrolling

Generic dimension loops prevent compiler optimizations such as instruction pipelining and register allocation. We provide manual unrolled implementations for 2D and 3D cases using `if constexpr`, enabling:

- Elimination of loop overhead (counter increment, bounds check)
- Instruction-level parallelism through independent operations
- Better register allocation for intermediate values
- Opportunities for SIMD auto-vectorization

This contributes 5-10% improvement and combines multiplicatively with reciprocal optimization.

3.2.3 Optimized Morton Encoding

The bit-spreading technique for Morton encoding replaces nested loops with a fixed sequence of shift and mask operations, providing 5-8% improvement on build operations where hash computation is called for every entity.

3.2.4 Swap-and-Pop Removal

Unordered removal using swap-and-pop provides $O(1)$ element removal instead of $O(m)$ for `std::remove`, contributing 3-5% improvement on incremental updates.

3.2.5 Query Radius Optimization

Applying reciprocal multiplication to cell radius computation:

$$r_i = \lceil r \cdot s_i^{-1} \rceil \quad (7)$$

provides 20-30% improvement on query operations by eliminating repeated divisions.

3.3 Memory Efficiency

Memory consumption consists of:

- Hash table overhead: approximately 24-32 bytes per occupied cell (platform dependent)
- Entity indices: 8 bytes per entity per cell (for `std::size_t`)
- Tracking vector: $n \times d \times 4$ bytes for n entities in d dimensions
- Configuration: $O(d)$ bytes (negligible)

For 10 million particles in 3D with average occupancy of 3 entities per cell and 3.3 million occupied cells:

$$\begin{aligned} \text{Hash table} &\approx 3.3M \times 28 \text{ bytes} = 92 \text{ MB} \\ \text{Entity indices} &\approx 10M \times 8 \text{ bytes} = 80 \text{ MB} \\ \text{Tracking} &\approx 10M \times 3 \times 4 \text{ bytes} = 120 \text{ MB} \\ \text{Total} &\approx 292 \text{ MB} \end{aligned}$$

This represents 60,000x reduction compared to a dense grid for the same world size.

4 Performance Evaluation

4.1 Benchmark Methodology

We evaluated performance using Google Benchmark on a 12-core processor at 4.39 GHz with 16 MB L3 cache. Benchmarks use fixed random seeds for reproducibility and report median times over multiple iterations to account for variance.

Test scenarios include:

- Build operations: 1K, 10K, 100K entities in 3D

- Incremental updates: 10K entities with 1% cell changes
- Radius queries: small (20 units), medium (50 units), large (100 units)
- Pair processing: 5K entities with 30-unit interaction radius
- Topology comparison: bounded, toroidal, infinite modes

All benchmarks use a 1000^3 world with 10-unit cells in 3D or 1000^2 world with 10-unit cells in 2D, representing typical game engine or simulation parameters.

4.2 Build Performance

Table 1 shows grid construction performance across entity counts:

Table 1: Grid build performance (3D, bounded topology, v1.1.0 with small vector optimization)

Entity Count	Time	Throughput	Per-Entity
1,000	65 μ s	15.4M entities/s	65 ns
10,000	407 μ s	24.6M entities/s	41 ns
100,000	3.38 ms	29.6M entities/s	34 ns

Throughput increases with entity count due to amortization of hash table initialization costs and better cache utilization from spatial locality. The 100K entity case achieves 29.6M entities/second throughput, or approximately 34 nanoseconds per entity including hash computation, bucket insertion, and inline storage. The small vector optimization provides the largest benefit on build operations by eliminating heap allocations for the majority of cells.

4.3 Incremental Update Performance

Figure ?? compares incremental updates against full rebuilds:

Table 2: Update performance (10K entities, 1% cell changes, v1.1.0)

Method	Time	Speedup
Full rebuild	407 μ s	1.0x
Incremental update	16 μ s	25.4x

Incremental updates achieve 25x speedup when only 1% of entities change cells. While this is lower than the v1.0 result (40x), it reflects that both operations benefit from small vector optimization, with rebuild improving more dramatically. This optimization remains critical for real-time applications maintaining 60 FPS where frame budgets are 16.7 milliseconds.

4.4 Query Performance

Table 3 presents radius query performance for different query radii:

Query time scales with the number of cells in the bounding box. For the medium radius (50 units, 5x cell size), the grid achieves 400K queries per second. The reciprocal optimization contributes significantly here, as cell radius computation occurs once per query.

Table 3: Radius query performance (10K entities)

Radius	Cells Checked	Time	Throughput
20 units	27 cells	1.8 μ s	556K queries/s
50 units	125 cells	2.5 μ s	400K queries/s
100 units	1000 cells	8.2 μ s	122K queries/s

4.5 Pair Processing Performance

Collision detection using `for_each_pair` achieves 2.4M pairs/second for 5K entities with 30-unit interaction radius. Compared to the naive $O(n^2)$ approach on 1K entities:

Table 4: Pair processing comparison

Method	Entities	Throughput
Naive $O(n^2)$	1,000	125K pairs/s
Spatial hash	5,000	2.4M pairs/s

The spatial hash scales to 5x more entities while achieving 19x higher throughput, demonstrating the importance of spatial partitioning for collision detection.

4.6 Topology Comparison

Table 5 shows build performance across topology modes:

Table 5: Topology mode comparison (10K entities, v1.1.0)

Topology	Time
Bounded	407 μ s
Toroidal	419 μ s
Infinite	404 μ s

Performance differences between topologies are negligible (under 4%), indicating that the switch-based topology selection introduces minimal overhead. The compiler likely optimizes the switch to direct jumps.

4.7 Optimization Impact

Table 6 quantifies the cumulative effect of optimizations:

The small vector optimization provides the largest single improvement by eliminating heap allocations for typical cell sizes (≤ 16 entities). Combined with micro-optimizations achieving 23% geometric mean improvement, the v1.1.0 implementation demonstrates 40-66% overall improvement over the baseline, with the largest gains on build operations.

Table 6: Optimization impact summary (v1.1.0)

Optimization	Improvement
Small vector (inline storage)	5-40%
Precomputed reciprocals	8-30%
Manual loop unrolling	5-10%
Optimized Morton encoding	5-8%
Swap-and-pop removal	3-5%
Query radius optimization	20-30%
Micro-opt geometric mean	23%
Combined improvement	40-66%

5 Applications and Use Cases

5.1 Game Development: Collision Detection

Game engines require efficient broad-phase collision detection to identify potentially colliding object pairs before expensive narrow-phase tests. A typical implementation:

```

1 // Setup collision grid
2 grid_config<3> cfg{
3     .cell_size = {5.0f, 5.0f, 5.0f},
4     .world_size = {1000.0f, 1000.0f, 1000.0f},
5     .topology_type = topology::bounded
6 };
7 sparse_spatial_hash<GameObject, 3> collision_grid(cfg);
8
9 // Each frame
10 collision_grid.update(game_objects);
11
12 std::vector<CollisionPair> collisions;
13 collision_grid.for_each_pair(game_objects, collision_radius,
14     [&](std::size_t i, std::size_t j) {
15         if (narrow_phase_test(game_objects[i], game_objects[j])) {
16             collisions.push_back({i, j});
17         }
18     });

```

Listing 8: Game collision detection example

The incremental update allows maintaining 60 FPS with thousands of dynamic objects by avoiding full grid reconstruction each frame.

5.2 Molecular Dynamics: Neighbor Lists

Molecular dynamics simulations compute forces between nearby atoms. The Verlet neighbor list algorithm maintains lists of atom pairs within a cutoff distance:

```

1 grid_config<3> cfg{
2     .cell_size = {cutoff, cutoff, cutoff},
3     .world_size = {box_x, box_y, box_z},
4     .topology_type = topology::toroidal // Periodic boundaries

```

```

5 };
6 sparse_spatial_hash<Atom, 3, double> grid(cfg);
7
8 // Update neighbor lists every N steps
9 if (step % neighbor_update_interval == 0) {
10     grid.rebuild(atoms);
11 }
12
13 // Compute forces
14 grid.for_each_pair(atoms, cutoff,
15     [&](std::size_t i, std::size_t j) {
16         Vector3 force = compute_lennard_jones(atoms[i], atoms[j]);
17         atoms[i].force += force;
18         atoms[j].force -= force; // Newton's third law
19     });

```

Listing 9: Molecular dynamics neighbor list

The toroidal topology correctly handles periodic boundaries, essential for simulating bulk material properties with finite computational domains.

5.3 Computational Geometry: Proximity Queries

Spatial databases and geographic information systems answer proximity queries such as finding all points of interest within a given radius:

```

1 grid_config<2> cfg{
2     .cell_size = {1000.0, 1000.0}, // 1km cells
3     .world_size = {100000.0, 100000.0}, // 100km x 100km
4     .topology_type = topology::bounded
5 };
6 sparse_spatial_hash<Location, 2, double> spatial_index(cfg);
7
8 spatial_index.rebuild(locations);
9
10 // Find all locations within 5km of query point
11 auto nearby = spatial_index.query_radius(5000.0, query_x, query_y);
12
13 for (std::size_t idx : nearby) {
14     if (euclidean_distance(locations[idx], query) <= 5000.0) {
15         results.push_back(locations[idx]);
16     }
17 }

```

Listing 10: GIS proximity query example

5.4 When to Use Sparse Spatial Hash

The sparse spatial hash excels when:

- Entities undergo frequent small movements (incremental updates provide large speedup)
- The world is large and sparsely populated (sparse storage critical)

- Radius queries dominate over k-nearest neighbor queries
- Uniform cell sizes suit the data distribution
- Periodic boundaries are required (toroidal topology)

Alternative structures may be preferable when:

- Data is static or infrequently updated (R-tree, k-d tree)
- Non-uniform spatial distribution requires adaptive resolution (octree, R-tree)
- k-nearest neighbor queries dominate (k-d tree)
- The world is small and densely populated (dense grid)

6 Related Work

Spatial hashing for collision detection. Teschner et al. [1] introduced optimized spatial hashing for collision detection in deformable objects, demonstrating significant performance improvements over hierarchical structures. Our work extends this approach with generic programming, incremental updates, and topology modes.

Hierarchical spatial structures. Quadrees, octrees, and R-trees provide adaptive resolution but require rebalancing or reconstruction for dynamic scenes. Boost.Geometry [2] provides an R-tree implementation suitable for static or semi-static data. Our sparse hash offers better performance for highly dynamic scenarios through incremental updates.

Dense uniform grids. Green [3] describes uniform grid collision detection for GPU-accelerated particle systems. While providing excellent performance for dense distributions, memory consumption becomes prohibitive for sparse large-scale simulations.

Z-order curves and space-filling curves. Morton ordering for spatial locality has been extensively studied in database systems [4] and graphics [5]. We apply this technique to hash-based spatial indexing to improve cache utilization.

Generic programming in C++. Stepanov and McJones [6] established principles for generic programming emphasizing zero-overhead abstractions. We apply these principles using modern C++20 features including concepts, `if constexpr`, and ranges.

7 Conclusions and Future Work

7.1 Summary

We presented `boost_sparse_spatial_hash`, a generic N-dimensional sparse spatial hash grid designed for high-performance neighbor search in dynamic scenes. The library achieves:

- 29.6M entities/second build throughput for 100K entities
- 25x speedup for incremental updates with 1% cell changes
- 400K queries/second for medium-radius proximity queries
- 2.4M pairs/second for collision detection

- 60,000x memory reduction compared to dense grids for sparse worlds
- 5-40% improvement from small vector optimization
- 23% additional improvement from micro-optimizations

The design leverages C++20 concepts, compile-time polymorphism, and position accessor traits to provide zero-overhead abstractions while maintaining extensibility and type safety. A custom small vector implementation stores typical cell sizes (≤ 16 entities) inline, eliminating heap allocations for 80-90% of cells. Targeted micro-optimizations including precomputed reciprocals, manual loop unrolling, and optimized Morton encoding provide additional performance gains.

Three topology modes (bounded, toroidal, infinite) support diverse application domains including game development, molecular dynamics, particle simulations, and computational geometry. The header-only implementation requires only the C++20 standard library and targets Boost-quality standards. The library has been rigorously validated with 54 test cases covering 326 assertions.

7.2 Boost Submission Status

The library (v1.1.0) is ready for submission to Boost:

- **Documentation:** Complete MkDocs site with API reference, tutorials, and design rationale
- **Testing:** 54 comprehensive test cases covering 326 assertions (100% pass rate)
- **Quality:** Exception safety guarantees verified, Morton encoding limits tested
- **Portability:** C++20 compliant (GCC 10+, Clang 12+, MSVC 2019+)
- **Performance:** Production-tested on 10M+ particle simulations

7.3 Future Directions

Several optimization opportunities remain:

SIMD vectorization. Processing multiple entities simultaneously using AVX2/AVX-512 could provide 20-30% improvement on build and update operations. The challenge is maintaining the generic interface while providing SIMD fast paths for specific types.

Parallel construction. Multi-threaded grid building using thread-local cell maps with a merge phase could achieve 2-4x speedup on large datasets. Load balancing and merge overhead require careful tuning. Early experiments with parallel pair processing showed atomic contention issues, suggesting this approach requires careful design.

Flat hash map. Replacing `std::unordered_map` with `boost::unordered_flat_map` provides better cache locality through open addressing, with expected 10-15% improvement.

Hilbert curve ordering. Hilbert curves provide better spatial locality than Morton ordering but require more complex encoding. This may yield 5-10% query improvement.

GPU acceleration. A CUDA or SYCL backend could enable massive parallelism for large-scale simulations with millions of entities, though this requires careful consideration of memory transfer costs.

Adaptive cell sizing. Supporting non-uniform cell sizes through hierarchical grids or spatial subdivision could improve performance for non-uniform distributions while maintaining the sparse hash advantages.

Reverse mapping optimization. Early experiments with $O(1)$ entity lookup showed that indirection overhead outweighed benefits for typical cell sizes ($n \leq 10$). However, for applications with very large cells ($n > 50$), a hybrid approach with conditional reverse mapping may prove beneficial.

7.4 Availability

The library is available at https://github.com/spinoza/sparse_spatial_hash under the Boost Software License 1.0. It includes comprehensive unit tests, performance benchmarks, and example applications.

References

- [1] M. Teschner, B. Heidelberger, M. Müller, D. Pomeranets, and M. Gross, “Optimized Spatial Hashing for Collision Detection of Deformable Objects,” *Proc. Vision, Modeling, Visualization*, 2003, pp. 47-54.
- [2] Boost.Geometry Library, “R-tree spatial index,” <https://www.boost.org/doc/libs/release/libs/geometry/>, accessed 2025.
- [3] S. Green, “Particle Simulation using CUDA,” NVIDIA Technical Report, 2008.
- [4] G. M. Morton, “A Computer Oriented Geodetic Data Base and a New Technique in File Sequencing,” IBM Technical Report, 1966.
- [5] M. Pharr and R. Fernando, “GPU Gems 2: Programming Techniques for High-Performance Graphics and General-Purpose Computation,” Addison-Wesley Professional, 2005.
- [6] A. Stepanov and P. McJones, “Elements of Programming,” Addison-Wesley Professional, 2009.
- [7] L. Verlet, “Computer ‘Experiments’ on Classical Fluids. I. Thermodynamical Properties of Lennard-Jones Molecules,” *Physical Review*, vol. 159, no. 1, pp. 98-103, 1967.
- [8] C. Ericson, “Real-Time Collision Detection,” Morgan Kaufmann, 2004.
- [9] H. Samet, “The Design and Analysis of Spatial Data Structures,” Addison-Wesley, 1990.